

CS837: ADVANCED OPERATING SYSTEMS

Operating System Overview & Core Foundations

Dr. Sarosh Tahir

sarosh.tahir@seecs.edu.pk

School of Electrical Engineering and Computer Science (SEECS)
National University of Sciences and Technology (NUST)

Lectures combine slides and board discussions to reinforce key OS concepts

Administration & Philosophy

- **Interplay of Methods:** Classes will actively combine **slides and board work** to illustrate theoretical concepts and mathematical proofs.
- **LMS Accessibility:** Core lecture slides will be posted regularly on the LMS platform.
- **No Scribe Policy:** Scribes are not officially provided. Students are expected to **take their own notes** during active board discussions.

Slide Updates & Pre-requisites

- **Dynamic Updates:** Slides are **living documents**. The instructor may update or correct them at any stage of the semester.
- **Best Practice:** Always download the latest version of the slides from LMS immediately prior to each class.
- **Quotes & Inspiration:** *I hear and I forget. I see and I remember. I do and I understand.* — Confucius

Operating systems manage shared hardware resources and bridge the user-to-hardware gap

"Operating systems are those programs that interface the machine with the applications programs. The main function of these systems is to dynamically allocate the shared system resources to the executing programs... concerned with the management and scheduling of memory, processes, and other devices..."

— WHAT CAN BE AUTOMATED?: THE COMPUTER SCIENCE AND ENGINEERING RESEARCH STUDY,
MIT Press, 1980

Key takeaway: An operating system is fundamentally both a software intermediary that abstracts hardware, and an allocator responsible for managing resource contention among competing applications.

Operating systems provide simplified abstractions and manage critical resources like memory

Three Core Pillars of an OS

- **Simplifying Intermediary:** Programs that make it fundamentally easier and more efficient to execute software applications.
- **Abstractions Provider:** Presents simple, elegant virtual abstractions of physical hardware, shielding programs from low-level register/timing complexities.
- **Resource Allocator:** Resolves conflict between multi-user resource demands, scheduling execution and managing storage.

OS Classification & Industry Examples

- **Desktop & Servers:**

Windows XP/2000/10/11, Ubuntu Linux, FreeBSD, Fedora, Solaris, macOS

- **Mobile Operating Systems:**

Android, iOS, Symbian OS, Lynx OS

- **Legacy Environments:**

DOS, IBM OS/2

Convenience, efficiency, and adaptability define the core objectives of modern operating systems

1. CONVENIENCE

- **User Centricity**

Makes computers more accessible and intuitive for human operators.

- **Developer Leverage**

Provides high-level APIs so programmers don't write device-specific driver codes.

2. EFFICIENCY

- **Asset Maximization**

Utilizes physical CPU cycles, RAM blocks, and storage blocks in an optimal, non-wasteful manner.

- **Throughput Scaling**

Manages process overhead to run more jobs per second.

3. ABILITY TO EVOLVE

- **Future Proofing**

Designed to seamlessly integrate new hardware features, modules, and upgrades without system disruption.

- **Continuous Improvement**

Allows runtime patching for security flaws, bug fixes, and new services.

The system infrastructure spans hardware, OS, libraries, and application software layers

Application Programs

Application Programming Interface (API) Boundary

Libraries & Utilities

Application Binary Interface (ABI) Boundary

Operating System

Instruction Set Architecture (ISA) Boundary

Execution Hardware

System Infrastructure Elements

- **Application Programs:** The topmost layer where processes run to solve user-specified domain tasks. Executes in protected user space.
- **Libraries & Utilities:** Translates source API commands into runtime-enforceable binary ABI instructions.
- **Operating System:** Intermediary kernel space software which direct operations of underlying CPUs, registers, and memories.
- **Execution Hardware:** Physical execution silicon, memory buses, caches, network cards, and secondary platters.

An operating system delivers seven vital services including program execution and file access

Development & Execution Services

- **Program Development:** Offers utility tools (linkers, editors, debuggers) to streamline software development. Often packaged with OS platforms.
- **Program Execution:** Automates program loading into memory, schedules hardware instruction blocks, and configures physical execution context.
- **Access I/O Devices:** Standardizes hardware calls so developers can use abstract commands, hiding physical device controller register complexity.
- **Controlled File Access:** Governs structured file storage, implements directory structures, and enforces device physical media mapping.

Protection & System Management

- **System Access:** Arbitrates shared resource access in multi-user networks, protecting storage partitions and enforcing user credentials.
- **Error Detection & Response:** Intercepts hardware-level faults (parity errors, device failures) and software-level crashes (arithmetic traps, page violations).
- **Accounting:** Dynamically monitors resource utilization metrics for performance profiling, capacity planning, and server billing.

Three key interfaces—ISA, ABI, and API—govern system-to-hardware communication

Defining Interface Boundaries

- **1. Instruction Set Architecture (ISA)**

Defines the literal machine language instruction repertoire (registers, opcodes, memory modes) supported by physical silicon.

- **2. Application Binary Interface (ABI)**

Establishes binary standards for program interaction across libraries and OS. Governs register conventions and runtime constraints.

- **3. Application Programming Interface (API)**

Defines source-level function calling signatures, data types, and interface contracts for high-level language software.

Interface Migration Dynamics

- **Boundary Shifting:** The boundaries between interfaces are fluid and shift over time based on research and performance trends.

- **Hardware Migration:** Frequently, functions that were originally purely implemented in OS software migrate directly into the hardware chip design.

- **Software Migration:** Extraneous user library codes often get directly absorbed into core operating system modules to improve efficiency.

APIs govern source code compatibility, while ABIs define the binary runtime interaction

API: Application Programming Interface

- **Source Code Contract:** Establishes standard interfaces for source-level code. Developers write programs targeting a specific API.
- **Build-Time Enforced:** Evaluated during compilation by the compiler which checks types and function signatures.
- **Hardware Independent:** Keeps high-level source portable across varying microarchitectures.

ABI: Application Binary Interface

- **Binary Link Contract:** Enforces rules governing compiled machine code compatibility.
- **Runtime Enforced:** Set by the linker and OS. Dictates register-passing rules, stack layouts, and system call opcodes.
- **Environment Bound:** Specific to a operating system kernel and physical CPU architecture (e.g. Linux x86_64 ABI).

The operating system controls system resources from within, unlike external control mechanisms

Traditional Control (External)

- **External Entity:** Traditional control structures sit entirely outside the system they control.
- **Passive Monitored:** Controls the system from an isolated environment, utilizing independent wiring.
- **Example: Heating & Thermostat:** A thermostat sits on the wall, completely independent of the furnace heating elements and boiler tanks.

OS Control (Internal Software)

- **Internal Soft Entity:** An OS is not external hardware; it is standard software executed by the processor.
- **Resource Competitor:** Shares memory, shares storage buses, and competes for CPU cycles on the same chip as user tasks.
- **Cooperative Preemption:** Relinquishes processor control to user programs and must depend on hardware interrupts to regain control.

The OS functions as standard software but frequently relinquishes control to user programs

Software Execution Profile

- **Executable Suite:** At its core, an OS is a program (or suite of programs) written in low-level languages (C, Assembly) and executed directly on the physical processor.
- **Memory Footprint:** Partially resident in main memory. Non-resident utilities are loaded from secondary storage on demand.
- **Privilege Boundary:** Executes instructions in high-privilege kernel mode while application processes execute in low-privilege user mode.

The Yield & Reclaim Mechanism

- **Control Relinquishment:** The OS does not run continuously. It starts a user process, sets hardware registers, and yields CPU execution.
- **Processor Dependence:** Once yielded, the OS depends entirely on processor hardware structures to regain execution control.
- **Interrupt Loop:** Hardware events (timer ticks, I/O completion, system traps) force the CPU to context-switch back into OS kernel handlers.

The operating system dynamically allocates CPU, memory, and I/O resources to active jobs

PROCESSOR (CPU) MANAGEMENT

- Schedules active processes.
- Manages context switches.
- Controls quantum slices to optimize CPU utilisation.

MAIN MEMORY ALLOCATION

- Maps virtual memory page structures.
- Allocates physical RAM block partitions.
- Controls page sharing and memory boundaries.

SECONDARY STORAGE CONTROL

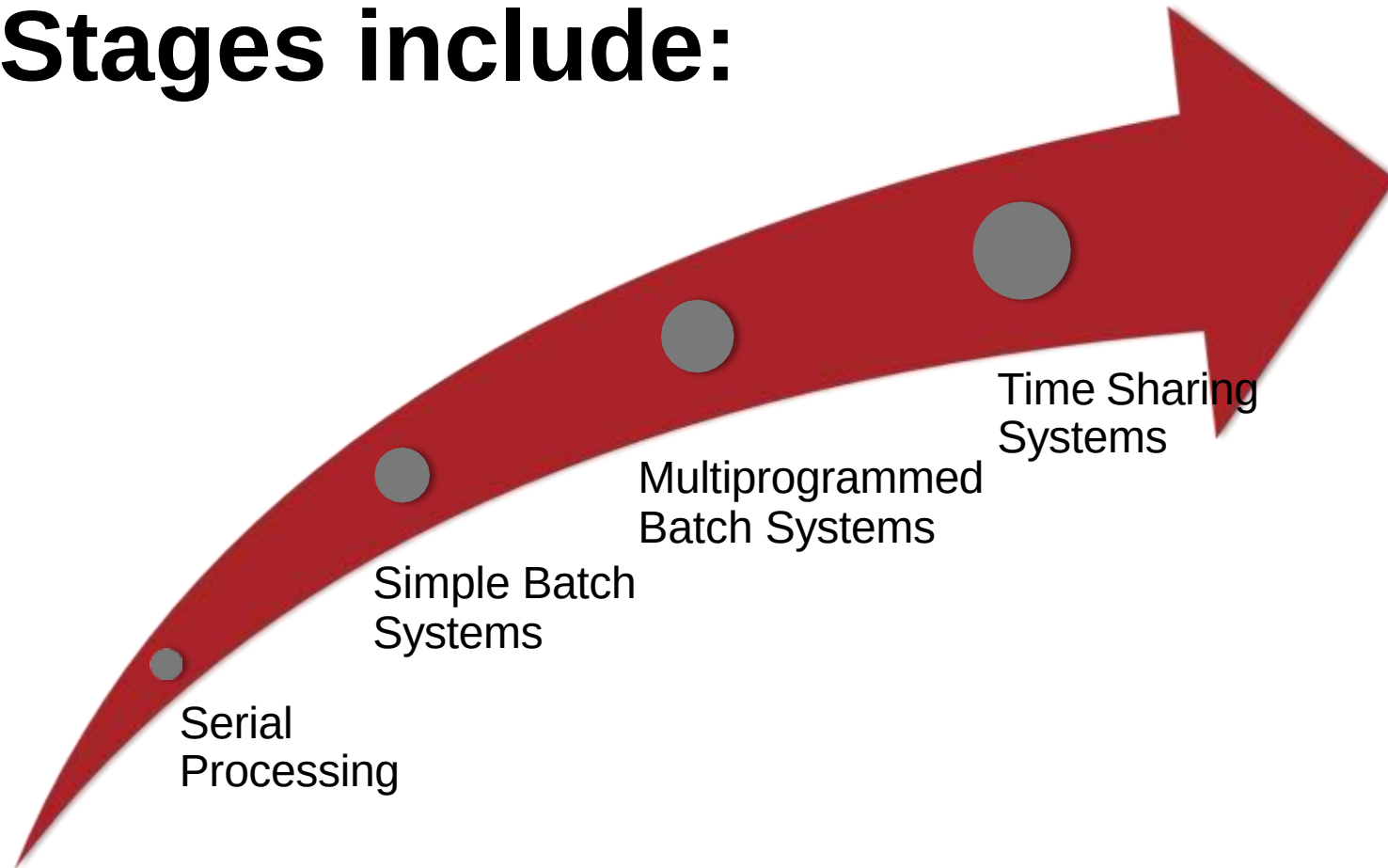
- Organizes folders and file indices.
- Maps storage blocks on physical platters.
- Schedules disk access requests to minimize seek times.

I/O DEVICES ARBITRATION

- Handles buffering, caching, and queuing.
- Schedules device-level data transfer cycles.
- Arbitrates concurrent physical device demands.

EVOLUTION OF OPERATING SYSTEMS

- **Stages include:**



OS architectures evolve over time across four main historical stages to support upgrades

Triggers for System Evolution

- **Hardware Upgrades:** Adapting kernel structures to support larger memories, newer bus architectures, and peripheral developments.
- **New Types of Hardware:** Ingesting abstractions for virtual machine monitors, graphics co-processors (GPGPUs), and high-speed SSD storage.
- **New Services:** Extending OS layers to native network stacks, security policies, and containerization frameworks.
- **Bug Fixes:** Rectifying kernel bugs and addressing security vulnerabilities dynamically.

Four Historical Eras

- **1. Serial Processing (1940s-1950s):** No operating system; users interact directly with raw switches and display lights.
- **2. Simple Batch Systems (1950s):** Resident monitor processes sequential magnetic tape or punched card batches.
- **3. Multiprogrammed Batch Systems:** Memory holds multiple jobs to overlap CPU and I/O utilization.
- **4. Time-Sharing Systems:** Interleaves interactive processes via clock interrupts (quanta).

Early serial processing systems lacked an OS, forcing direct and inefficient human-hardware interaction

System Characteristics

- **No Software Layer:** Completely devoid of operating systems. Programmers interact directly with raw hardware boards.
- **Console Operation:** Programs run manually from a hardwired console with display lights, physical toggle switches, card readers, and printers.
- **Hardcopy Scheduling:** Users sign up on a paper reservation sheet to gain exclusive system access for a fixed hours slot.

Major Inefficiencies

- **Wasted CPU Time:** If a user finishes early, the computer sits completely idle. If a programmer runs over time, they must stop.
- **Setup Bottlenecks:** Significant time is wasted on manual setup—loading the compiler, assembling source code, binding files, loading runtime libraries.
- **Brittle Debugging:** Programs are debugged directly on the console. If a memory address goes wrong, the programmer stops and restarts from scratch.

Resident monitors control job execution sequence to automate early batch systems

The Concept: Resident Monitor

- **The Punched Card Batch:** Computer operators batch programs (jobs) together onto high-speed magnetic tape devices.
- **Always Resident:** The Resident Monitor is a basic software program that always resides in physical main memory.
- **Automatic Sequencing:** The monitor reads a job from the input device, loads it to user memory, executes it, and on completion, loads the next job.

Monitor Control Flow

- **Monitor Perspective:** The monitor controls the entire sequence of events. When a user program finishes or hits an error, control is returned to the monitor.
- **Processor Perspective:** CPU executes instructions from monitor space. When the monitor yields control, the CPU fetches user instructions until termination occurs.
- **Setup Overhead Reduction:** Eliminates idle CPU periods between manual human handoffs.

MONITOR POINT OF VIEW

- ❖ Monitor controls the sequence of events
- ❖ **Resident Monitor** is software that always resides in memory
- ❖ Monitor reads in job and gives control
- ❖ Job returns control to monitor

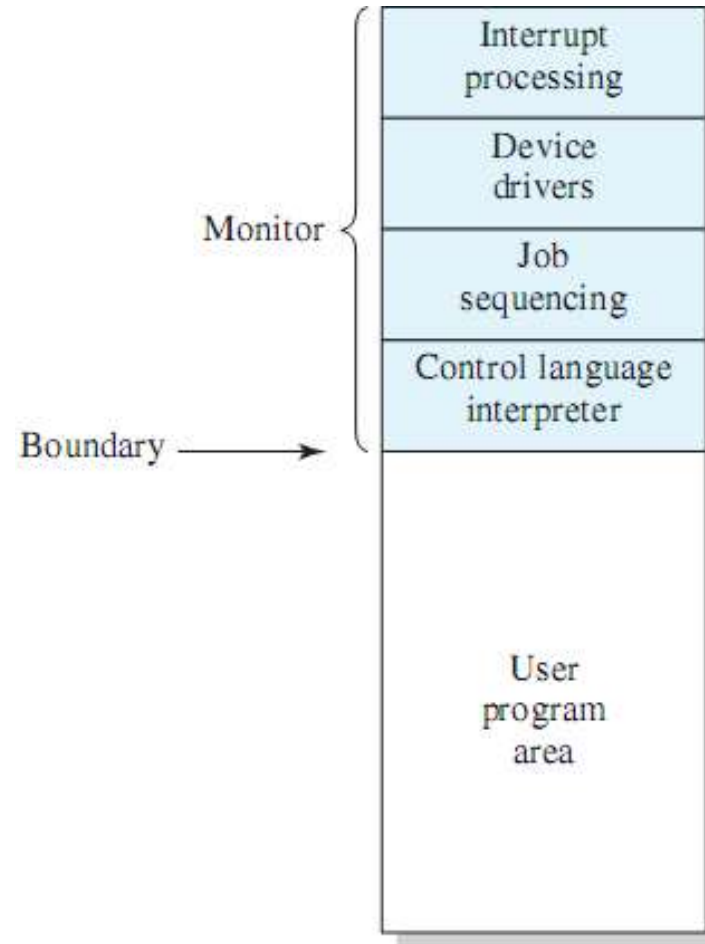


Figure 2.3 Memory Layout for a Resident Monitor

Four key hardware features—including memory protection and timers—support stable OS control

Memory Protection & Timers

- **Memory Protection:**

The user program must never alter or corrupt the memory partitions containing the resident monitor.

Enforced by hardware check registers. If a program attempts a boundary violation, it generates a hardware trap.

- **Hardware Timers:**

Prevents any single user job from monopolizing the shared system indefinitely (e.g. infinite loops).

The OS sets a hardware countdown timer. When the timer hits zero, the physical processor triggers an interrupt, context-switching control back to the monitor.

Privileged Instructions & Interrupts

- **Privileged Instructions:**

Critical instructions (like direct hardware I/O or register modifications) can only be executed by the resident monitor.

If a user program attempts to run a privileged command, the CPU blocks it and traps to the OS.

- **Interrupt Handling Hardware:**

Provides the OS with architectural flexibility to suspend running code and manage asynchronous devices.

Directly routes device signals back into registered kernel handler spaces.

Two operating modes—user and kernel—protect critical memory from unauthorized access

User Mode (Unprivileged)

- **Application Domain:** Standard user applications execute in this restricted environment.
- **Memory Shielding:** Access to physical OS memory addresses is blocked by hardware boundary registers.
- **Blocked Instructions:** Attempting to execute privileged instructions (like direct physical I/O or modifying control registers) triggers a CPU trap.

Kernel / Supervisor Mode (Privileged)

- **OS Kernel Domain:** The resident monitor and operating system core run in this fully privileged environment.
- **Raw Hardware Access:** All privileged instructions are active, allowing full, direct control over all CPU registers and physical components.
- **Complete Memory Access:** Enables the kernel to view and modify any address in both user program space and kernel space.

Multiprogramming maximizes resource utilization by interleaving multiple active jobs in memory

The Bottleneck: Uniprogramming

- **I/O Device Sluggishness:** Mainframe CPUs are incredibly fast, while mechanical card readers and tape drives are slow.
- **Wasted CPU Cycles:** Under uniprogramming, when a job reads/writes data, the CPU must block and sit idle waiting for the mechanical operation to finish.
- **Sub-Optimal Utilization:** Resulted in processor utilization rates as low as **20%** on expensive hardware.

The Solution: Multitasking

- **Expanded Memory:** Main memory is scaled up to hold the Resident Monitor plus multiple user programs concurrently (e.g. Program A, B, C).
- **Processor Interleaving:** When Program A initiates an I/O wait, the CPU immediately context-switches to execute active instructions in Program B.
- **Shared Scheduling:** By overlapping computation and mechanical transfers, overall resource utilization is dramatically multiplied.

MULTIPROGRAMMED BATCH SYSTEMS

Read one record from file	15 μ s
Execute 100 instructions	1 μ s
Write one record to file	<u>15 μs</u>
TOTAL	31 μ s

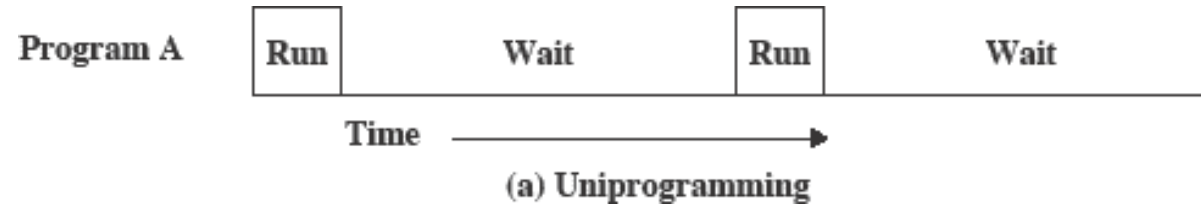
$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

Processor is often idle

- ❖ Even with automatic job sequencing
- ❖ I/O devices are slow compared to processor

Figure 2.4 System Utilization Example

UNIPROGRAMMING



- The processor spends a certain amount of time executing, until it reaches an I/O instruction; it must then wait until that I/O instruction concludes before proceeding

Multiprogramming doubles processor utilization and slashes job elapsed times in half

Performance Metric	Uniprogramming (1 Job)	Multiprogramming (3 Jobs)
Processor Utilization	20%	40% (Doubled)
Memory Utilization	33%	67% (Doubled)
Disk Utilization	33%	67% (Doubled)
Printer Utilization	33%	67% (Doubled)
Elapsed Time	30 Minutes	15 Minutes (Halved)
System Throughput	6 Jobs / Hour	12 Jobs / Hour (Doubled)
Mean Response Time	18 Minutes	10 Minutes (Reduced 44%)

Time-sharing systems partition processor time to support multiple concurrent interactive users

Interactive Multi-User Concept

- **Terminal Terminology:** Multiple concurrent human operators access the system simultaneously via physical interactive terminals.
- **Interactive Flow:** Unlike batch systems, users interact directly with running programs, entering commands and receiving rapid feedback.
- **The Illusion of Sole Control:** Interleaving process execution so rapidly that each terminal operator has the illusion of sole machine ownership.

Time-Slicing Mechanics

- **The Quantum Slice:** Processor time is partitioned into tiny execution intervals called quanta or time slices (e.g. 0.2 seconds).
- **Preemption Loop:** At each regular timer interrupt, the current process is preempted, written to memory/disk, and replaced with the next user task.
- **Context Overlap:** Optimizes user response time over batch processor throughput.

UTILIZATION HISTOGRAMS

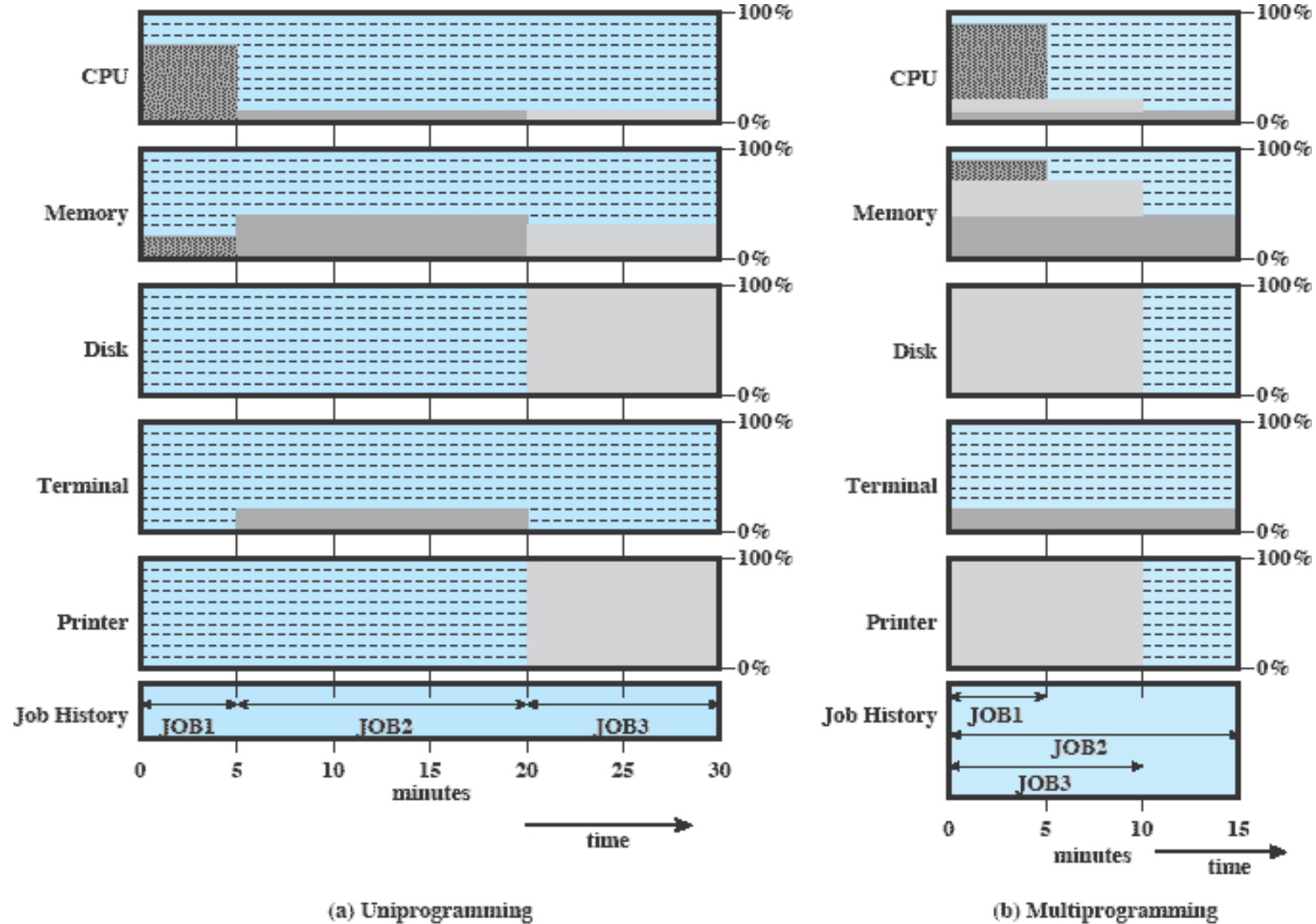


Figure 2.6 Utilization Histograms

Batch multiprogramming maximizes CPU use, whereas time-sharing minimizes user response times

Batch Multiprogramming Profile

- **Principal Objective:** Focuses strictly on maximizing processor utilization across sequential tasks.
- **Directive Ingestion:** Actions and system directives are loaded via **Job Control Language (JCL)** instructions batched with the programs.
- **Primary Focus:** High throughput processing of non-interactive jobs (e.g. mathematical simulations, file runs).

Time-Sharing Profile

- **Principal Objective:** Prioritizes minimizing user response times across multi-user sessions.
- **Directive Ingestion:** Operations are dynamically entered via **interactive terminal commands** at run-time.
- **Primary Focus:** Broad accessibility and active program interaction (e.g. text editing, code debugging, queries).

Early time-sharing systems like CTSS introduced time slicing with 0.2-second interrupts

CTSS Architecture (MIT Project MAC)

- **Hardware Footprint:** Ran on central mainframes with only **32,000 36-bit words** of main memory.
- **Resident Footprint:** The CTSS resident monitor occupied 5,000 words, leaving 27,000 words for user programs.
- **Boundary Standard:** To simplify memory translation, user programs were always loaded starting at the **5000th word** boundary.
- **Single Resident:** Due to size limits, only one user program was resident in memory at a time; others sat on magnetic disks.

Time-Slicing Execution Loop

- **Timer Interrupts:** Hardware clocks triggered interrupts at regular intervals of **0.2 seconds**.
- **Preemption & Save:** On interrupt, the OS preempted the current user and wrote their memory state out to a magnetic disk platter.
- **Load & Run:** The monitor then loaded the next scheduled user's program from disk into RAM starting at word 5,000, and resumed execution.
- **Optimized Platter Layout:** Used optimized cylinder blocks on disk to minimize magnetic head seek delay during swaps.

Five modern architectural approaches organize OS kernels to meet expanding system demands

1. MICROKERNEL

Assigns only essential operations to core kernel; moves others (file, net) to user space.

2. MULTITHREADED

Partitions a process execution context into concurrent, cooperative threads.

3. SMP SCALING

Schedules identical physical CPUs sharing memory concurrently.

4. DISTRIBUTED

Provides the illusion of single memory across clusters.

5. OBJECT-ORIENTED

Encapsulates modular extensions into structured kernel boxes.

Symmetric multiprocessing (SMP) scales performance and ensures high system availability

SMP Architecture Profile

- **Cooperative Silicon:** Multiple physical CPUs share access to unified main memory and shared I/O peripheral controllers.
- **Identical Capabilities:** Every processor is completely symmetrical, with identical capabilities to perform kernel work.
- **Unified Kernel:** A single operating system kernel runs across all processors, arbitrating job execution.

Major Advantages of SMP

- **Performance:** Multiplies execution speed by running distinct instruction streams on separate physical processors simultaneously.
- **System Availability:** Enhances fault tolerance. The failure of a single physical processor does not halt the OS; the scheduler shifts tasks to remaining CPUs.
- **Incremental Growth:** Performance can be scaled dynamically by adding processors to the physical system.
- **Scaling Range:** Vendor product lines can scale seamlessly from low-end single-processor configurations to massive multi-CPU servers.

Multiprocessor environments introduce five critical OS design challenges including synchronization

1. CONCURRENT PROCESSES

- Kernel routines must be designed to be completely **reentrant**.
- Enables multiple CPUs to execute the exact same kernel code simultaneously without race conditions.

2. SCHEDULING POLICIES

- Any physical processor may perform scheduling decisions.
- Complicates the task of enforcing unified process priorities and load-balancing schedules.

3. HARDWARE SYNCHRONIZATION

- Enforces synchronization over active processes sharing memory space.
- Prevents memory corruption from concurrent writes using hardware locks.

4. FAULT TOLERANCE

- Coordinates physical memory pages across physical processors.
- Graceful degradation: The OS must isolate failed CPUs and redirect active execution streams seamlessly.

Virtual machines virtualize architectures to run multiple guest operating systems simultaneously

Virtualization Concepts

- **Architectural Boundaries:** Virtualization inserts a software layer—the hypervisor or **Virtual Machine Monitor (VMM)**—between hardware and guest operating systems.
- **Instruction Isolation:** Guest operating systems believe they control physical execution registers, which the VMM virtualizes.
- **Resource Multiplexing:** Slices CPU execution slots and registers to run isolated guest OS kernels on a single computer.

Process vs. System Virtual Machines

- **Process Virtual Machines:**

Virtualizes code at the application process level, defining a high-level runtime machine using standard libraries (e.g., Java Virtual Machine, .NET CLR).

- **System Virtual Machines:**

Virtualizes the full underlying physical hardware, allowing a full guest operating system (e.g. Windows on Linux) to execute isolated inside a VM container.

Unix-like environments split operating systems into a privileged kernel and a user shell

The OS Kernel

- **Lowest Core Layer:** The low-level software block that communicates directly with execution hardware registers.
- **Privileged Execution:** Runs strictly in supervisor/kernel mode, with complete system instruction permissions.
- **Task Management:** Responsible for process context execution, main memory translation, physical device I/O schedules, and interrupts.

The Command Shell

- **User Interpreter:** A user-space utility program that reads commands from keyboard devices and initiates executing tasks.
- **Unprivileged Execution:** Runs completely in unprivileged user mode, accessing resources via the system call boundary.
- **Task Combination:** Unix shells allow users to link simple utilities via pipes to execute complex system workflows.

System calls provide a standardized gateway to request key OS services

System Call Interface & File Calls

- **The Service Gate:** The mechanism programs utilize to request kernel-mode services (I/O, memory, processes) from the OS.
- **CPU Transitions:** Standard user code executes system calls by triggering software interrupts, causing hardware to switch CPU context to kernel privilege mode.
- **Common File Management Calls:**
 - ``fd = open(file, how, ...)``: Request file handle
 - ``n = read(fd, buffer, bytes)``: Read file data to buffer
 - ``s = close(fd)``: Close active file descriptor

Process Management Calls

- ``pid = fork()``:
Creates a new child process that is an identical copy of the calling parent process. Both execute from the same instruction point, returning different PIDs.
- ``pid = waitpid(pid, &status, opt)``:
Suspends caller execution until a designated child process terminates.
- ``s = execve(name, argv, env)``:
Replaces the caller's process core memory image with a brand new compiled executable file.
- ``exit(status)``:
Terminates execution and returns exit status to parent.

Course material is grounded in textbook lectures and MIT CSE research studies

Grounding References

- **Core Material:** Grounded entirely in **01_1_Operating_System_Overview.pdf** (CS837 Advanced Operating Systems Overview).
- **Historical Definitions:** Grounded in MIT Press 1980 Research Study (*"What Can Be Automated?").
- **Case Studies:** CTSS historical timing and system dimensions verified through MIT Project MAC specifications.
- **Industry Interfaces:** Common ABI definitions reference stack growth, system call opcodes, and register alignment conventions.